

Міністерство освіти і науки України
Донецький національний університет імені Василя Стуса
Факультет інформаційних і прикладних технологій

Кафедра інформаційних технологій

ЗВІТ

З лабораторної роботи №15
дисципліни «Проектування СППР та систем управління»
на тему: «Аналіз статті та вирішення власної задачі на її основі»

Виконав: Молодченко Д. В.

Перевірив: Ротштейн О. П.

Вінниця 2025

Мета

Проаналізувати підхід зі статті щодо інтеграції нечіткої когнітивної карти (НKK) і принципу Беллмана–Заде для вибору сценарію керування. Сформулювати власну аналогічну задачу та реалізувати алгоритм у вигляді програми.

Короткий аналіз статті

У статті розглянуто вибір варіантів керування узагальненою динамічною системою в нечіткому середовищі. Для моделювання взаємовпливів керованих змінних і критеріїв застосовується НKK (матриця ваг W_0), а для узгодження багатьох критеріїв використовується принцип Беллмана–Заде (перетин нечітких множин).

Ключові ідеї підходу:

– динаміка концептів описується рекурентним співвідношенням:

$$x_i^{k+1} = x_i^k + \sum_{j=1..n} (x_j^k - x_j^{k-1}) \cdot w_{ji}.$$

– вводиться функція належності до «перфектності» для критерію:

$$\pi(x) = (x + 1)/2, \text{ де } x \in [-1; 1], \pi \in [0; 1].$$

– для кожного сценарію обчислюються ступені належності критеріїв, після чого формується рішення як перетин (Bellman–Zadeh):

$\mu_D(S_r) = \min(\mu_1(S_r), \mu_2(S_r), \dots, \mu_m(S_r))$, обирають S_r з максимальним μ_D .

Постановка власної задачі

Потрібно обрати найкращий сценарій керування для підвищення якості роботи домашнього серверу, на якому розміщено студентські веб-проекти (API/сайти). Враховуються керовані змінні U та критерії G , які взаємно впливають один на одного.

Перелік концептів НКК

№	Позначення	Зміст (інтерпретація)
1	C1	Рівень резервного живлення (UPS)
2	C2	Резервування Інтернет-каналу
3	C3	Моніторинг і алерти
4	C4	Оптимізація Nginx/кешування
5	C5	Планові техобслуговування
6	C6	Доступність сервісу (uptime)
7	C7	Швидкодія/затримка (latency)
8	C8	Задоволеність користувачів
9	C9	Витратна ефективність (чим більше – тим краще)

$U = \{C1, C2, C3, C4, C5\}$ – керовані змінні; $G = \{C6, C7, C8, C9\}$ – критерії оцінювання.

Матриця ваг W0

Ваги вибрано з дискретної шкали $(-1; -0.75; -0.5; -0.25; 0; 0.25; 0.5; 0.75; 1)$. У таблиці наведено матрицю W0 (рядок – джерело впливу C_j, стовпець – ціль C_i). Діагональні елементи дорівнюють 0.

C _j →C _i	C1	C2	C3	C4	C5	C6	C7	C8	C9
C1	0	0	0	0	0	0.75	0.25	0.25	-0.75
C2	0	0	0	0	0	0.75	0.5	0.25	-0.75
C3	0	0	0	0	0	0.5	0	0.5	0.5
C4	0	0	0	0	0	0.25	0.75	0.5	0.25
C5	0	0	0	0	0	0.5	0	0.25	-0.25
C6	0	0	0	0	0	0	0	0.75	0
C7	0	0	0	0	0	0	0	0.5	0
C8	0	0	0	0	0.25	0	0	0	0
C9	0	0	0.25	0	0.25	0	0	0	0

Сценарії керування

Сценарій	x1 (C1)	x2 (C2)	x3 (C3)	x4 (C4)	x5 (C5)
S1 (Економний базовий)	0.2	0.1	0.3	0.2	0.2
S2 (Збалансований)	0.5	0.4	0.5	0.5	0.4
S3 (Фокус на швидкодію)	0.4	0.3	0.5	0.9	0.4
S4 (Фокус на	0.9	0.8	0.6	0.5	0.7

надійність)					
S5 (Моніторинг + ТО)	0.6	0.3	0.9	0.4	0.9
S6 (Ульт्रा-дешевий)	0.2	0.1	0.1	0.1	0.1
S7 (Преміум)	0.9	0.9	0.9	0.9	0.9
S8 (Розумна оптимізація)	0.3	0.3	0.9	0.9	0.4

Алгоритм

Крок 1. Задати початковий стан X^0 : керовані змінні $x_1..x_5$ приймають значення сценарію, а критерії $x_6..x_9 = 0$.

Крок 2. Ітераційно обчислити стаціонарний стан X^1 за рекурентною формулою:

$$X^{k+1} = X^k + (X^k - X^{k-1}) \cdot W^0, \quad k = 0, 1, 2, \dots$$

(для $k=0$ використовується $X^{-1} = 0$, тому $X^1 = X^0 + X^0 \cdot W^0$).

Крок 3. Нормалізувати значення концептів до інтервалу $[-1; 1]$. Для точкових значень у програмі використано нормалізацію по кожному концепту за максимумом/мінімумом по всіх сценаріях (адаптація формули (8)).

Крок 4. Для критеріїв $C_6..C_9$ обчислити ступінь належності до перфектності $\pi(x) = (x+1)/2$.

Крок 5. Для кожного сценарію знайти $\mu_D = \min(\pi_6, \pi_7, \pi_8, \pi_9)$. Найкращий сценарій – з максимальним μ_D .

Результати

Таблиця містить ступені перфектності критеріїв (μ_6 .. μ_9) та інтегральну оцінку μ_D для кожного сценарію (чим більше – тим краще).

Сценарій	μ_6	μ_7	μ_8	μ_9	$\mu_D=\min$	іг .
S8 (Розумна оптимізація)	0.81234 5	0.83333 3	0.85285 3	0.47338 5	0.47338 5	31
S1 (Економний базовий)	0.61337 4	0.59259 3	0.61528 2	0.44401 6	0.44401 6	29
S6 (Ультра-дешевий)	0.56720 4	0.56481 5	0.56406 4	0.41446 4	0.41446 4	28
S3 (Фокус на швидкодії)	0.76719 5	0.84259 3	0.80397 1	0.37188 0	0.37188 0	31
S5 (Моніторинг + ТО)	0.85072 8	0.72222 2	0.83583 6	0.28322 3	0.28322 3	31
S2 (Збалансований)	0.75661 2	0.75925 9	0.76059 4	0.27936 9	0.27936 9	31
S7 (Преміум)	1.00000 0	1.00000 0	1.00000 0	0.02257 7	0.02257 7	31
S4 (Фокус на надійність)	0.89283 8	0.87037 0	0.87070 4	0.00000 0	0.00000 0	31

Найкращий сценарій за принципом Беллмана–Заде: S8 (Розумна оптимізація).

Значення $\mu_D = 0.473385$.

```

C:\cs\41\dss\15>lua lab15_fcm_bellman_zadeh.lua
Ранжування сценаріїв за принципом Беллмана–Заде ( $\mu_D = \min \mu_i$ ):
Сценарій           $\mu_6$        $\mu_7$        $\mu_8$        $\mu_9$        $\mu_D$       it
S8 (Розумна оптимізація)  0.812345  0.833333  0.852853  0.473385  0.473385  31
S1 (Економний базовий)   0.613374  0.592593  0.615282  0.444016  0.444016  29
S6 (Ульт्रा-дешевий)     0.567204  0.564815  0.564064  0.414464  0.414464  28
S3 (Фокус на швидкодiю)  0.767195  0.842593  0.803971  0.371880  0.371880  31
S5 (Моніторинг + T0)    0.850728  0.722222  0.835836  0.283223  0.283223  31
S2 (Збалансований)      0.756612  0.759259  0.760594  0.279369  0.279369  31
S7 (Преміум)            1.000000  1.000000  1.000000  0.022577  0.022577  31
S4 (Фокус на надійність) 0.892838  0.870370  0.870704  0.000000  0.000000  31

Найкращий сценарій: S8 (Розумна оптимізація)
 $\mu_D = 0.473385$ 

```

Рисунок 1. Скріншот запуску програми.

Висновок

Реалізовано підхід зі статті: побудовано НКК для задачі керування якістю роботи домашнього серверу та виконано вибір сценарію керування за принципом Беллмана–Заде. За отриманими результатами найкращий варіант – сценарій «Розумна оптимізація», оскільки він забезпечує найбільше значення μ_D , тобто найкраще «найгірше» значення серед усіх критеріїв.

Додаток А. Текст програми

Нижче наведено файл: lab15_fcm_bellman_zadeh.lua

-- Lab 15: Fuzzy scenario analysis using FCM + Bellman–Zadeh (Rotshtein et al.)

-- Run: lua lab15_fcm_bellman_zadeh.lua

local n = 9

local q = 5

local concept_names = {

 "C1 – Рівень резервного живлення (UPS)",

 "C2 – Резервування Інтернет-каналу",

 "C3 – Моніторинг і алерти",

 "C4 – Оптимізація Nginx/кешування",

 "C5 – Планові техобслуговування",

 "C6 – Доступність сервісу (uptime)",

"C7 – Швидкодія/затримка (latency)",
 "C8 – Задоволеність користувачів",
 "C9 – Витратна ефективність (чим більше – тим краще)",
 }

local W = {
 {0, 0, 0, 0, 0, 0.75, 0.25, 0.25, -0.75},
 {0, 0, 0, 0, 0, 0.75, 0.5, 0.25, -0.75},
 {0, 0, 0, 0, 0, 0.5, 0, 0.5, 0.5},
 {0, 0, 0, 0, 0, 0.25, 0.75, 0.5, 0.25},
 {0, 0, 0, 0, 0, 0.5, 0, 0.25, -0.25},
 {0, 0, 0, 0, 0, 0, 0, 0.75, 0},
 {0, 0, 0, 0, 0, 0, 0, 0.5, 0},
 {0, 0, 0, 0, 0.25, 0, 0, 0, 0},
 {0, 0, 0.25, 0, 0.25, 0, 0, 0, 0},
 }

local scenarios = {
 { name = "S1 (Економний базовий)", u = {0.2, 0.1, 0.3, 0.2, 0.2} },
 { name = "S2 (Збалансований)", u = {0.5, 0.4, 0.5, 0.5, 0.4} },
 { name = "S3 (Фокус на швидкодію)", u = {0.4, 0.3, 0.5, 0.9, 0.4} },
 { name = "S4 (Фокус на надійність)", u = {0.9, 0.8, 0.6, 0.5, 0.7} },
 { name = "S5 (Моніторинг + ТО)", u = {0.6, 0.3, 0.9, 0.4, 0.9} },
 }

```
{ name = "S6 (Ультра-дешевий)", u = {0.2, 0.1, 0.1, 0.1, 0.1} },  
{ name = "S7 (Преміум)", u = {0.9, 0.9, 0.9, 0.9, 0.9} },  
{ name = "S8 (Розумна оптимізація)", u = {0.3, 0.3, 0.9, 0.9, 0.4} },  
}
```

```
local function zeros(m)
```

```
    local v = {}
```

```
    for i = 1, m do v[i] = 0 end
```

```
    return v
```

```
end
```

```
local function vec_sub(a, b)
```

```
    local r = {}
```

```
    for i = 1, #a do r[i] = a[i] - b[i] end
```

```
    return r
```

```
end
```

```
local function vec_add(a, b)
```

```
    local r = {}
```

```
    for i = 1, #a do r[i] = a[i] + b[i] end
```

```
    return r
```

```
end
```

```
local function vec_mul_row_mat(v, M)
```

```
    local r = {}
```

```
    for j = 1, #M[1] do
```

```
        local s = 0
```

```
        for i = 1, #v do
```

```
            s = s + v[i] * M[i][j]
```

```
        end
```

```
        r[j] = s
```

```
    end
```

```
    return r
```

```
end
```

```
local function max_abs_diff(a, b)
```

```
    local m = 0
```

```
    for i = 1, #a do
```

```
        local d = math.abs(a[i] - b[i])
```

```
        if d > m then m = d end
```

```
    end
```

```
    return m
```

```
end
```

```
local function simulate_stationary(X0, eps, max_iter)
```

```
    local X_prev = zeros(n)
```

```

local X_curr = {}
for i = 1, n do X_curr[i] = X0[i] end
local it = 0
while it < max_iter do
    it = it + 1
    local delta = vec_sub(X_curr, X_prev)
    local influence = vec_mul_row_mat(delta, W)
    local X_next = vec_add(X_curr, influence)
    if max_abs_diff(X_next, X_curr) < eps then
        return X_next, it
    end
    X_prev, X_curr = X_curr, X_next
end
return X_curr, it
end

```

```

local function clamp(x, lo, hi)
    if x < lo then return lo end
    if x > hi then return hi end
    return x
end

```

-- Step 1: stationary vectors for all scenarios

```

local eps = 1e-9
local max_iter = 5000
local station = {}
for si = 1, #scenarios do
    local X0 = zeros(n)
    for i = 1, q do X0[i] = scenarios[si].u[i] end
    local Xl, it = simulate_stationary(X0, eps, max_iter)
    station[si] = { Xl = Xl, it = it }
end

```

-- Step 2: per-concept min/max over stationary states (adaptation of formula (8) for crisp values)

```

local max_pos = zeros(n)
local min_neg = zeros(n)
for i = 1, n do
    max_pos[i] = -math.huge
    min_neg[i] = math.huge
end
for si = 1, #scenarios do
    local Xl = station[si].Xl
    for i = 1, n do
        if Xl[i] > max_pos[i] then max_pos[i] = Xl[i] end
        if Xl[i] < min_neg[i] then min_neg[i] = Xl[i] end
    end
end

```

```

    end
end

local function normalize_x(i, x)
    if x > 0 then
        local d = max_pos[i]
        if d == 0 then return 0 end
        return clamp(x / d, -1, 1)
    elseif x < 0 then
        local d = min_neg[i] -- negative
        if d == 0 then return 0 end
        return clamp((-x) / d, -1, 1)
    else
        return 0
    end
end
end

```

```

-- membership to "perfection":  $\pi = (x_{\text{hat}} + 1)/2$ 

```

```

local function mu_perfection(x_hat)
    return (x_hat + 1) / 2
end

```

```

-- Step 3: Bellman–Zadeh aggregation (intersection):  $\mu_D = \min(\mu_i)$ 

```

```

local results = {}
for si = 1, #scenarios do
    local Xl = station[si].Xl
    local mu = {}
    local muD = 1
    for i = q + 1, n do
        local x_hat = normalize_x(i, Xl[i])
        local m = mu_perfection(x_hat)
        mu[i] = m
        if m < muD then muD = m end
    end
    results[si] = { name = scenarios[si].name, mu = mu, muD = muD, it =
station[si].it }
end

table.sort(results, function(a,b) return a.muD > b.muD end)

print('Ранжування сценаріїв за принципом Беллмана–Заде ( $\mu_D = \min \mu_i$ ):')
print(string.format('%-28s  $\mu_6$      $\mu_7$      $\mu_8$      $\mu_9$      $\mu_D$     it', 'Сценарій'))
for _, r in ipairs(results) do
    local mu6 = r.mu[6] or 0
    local mu7 = r.mu[7] or 0

```

```
local mu8 = r.mu[8] or 0  
local mu9 = r.mu[9] or 0  
  
print(string.format('%-28s %.6f %.6f %.6f %.6f %.6f %d', r.name, mu6,  
mu7, mu8, mu9, r.muD, r.it))  
end
```

```
local best = results[1]  
  
print("")  
print('Найкращий сценарій: ' .. best.name)  
print(string.format('μD = %.6f', best.muD))
```